
Five AI Technical Questions

EverCurrent Adaptive Daily Digest | Take-home written answers

1. How do you handle LLMs losing context in long-running or multi-step workflows?

I first decide what must survive between steps: source IDs, authorization, decisions already made, and the current user intent. I keep those as structured state outside the model instead of relying on a growing conversation transcript. Each step receives only the state and evidence it needs, returns a small validated result, and checkpoints any important decision. If a step fails, I can retry from that checkpoint without asking the model to reconstruct history.

For this digest, retrieval, grouping, ranking, and ACL are deterministic. The summarizer sees only the final authorized Top-K stories with source IDs and relationship evidence; its output is checked and cached against a source fingerprint. If I extended this into a longer workflow, I would add explicit step schemas, versioned state, and replay tests before adding more model context.

2. What if vector search returns nearly identical results with small but critical differences?

I treat similarity as a candidate-generation signal, not proof that two engineering messages say the same thing. A 24V result and a 48V result can be lexically almost identical yet imply very different decisions. I would filter by ACL and metadata first, union semantic candidates with keyword and exact-identifier hits, and compare typed numbers, units, revisions, dates, and part IDs before grouping facts.

This MVP uses deterministic TF-IDF plus exact-match union because the dataset is small. Its relationship rules keep duplicates, explicit replacements, and unresolved differences separate. M001/M002 are comparable measurements with different values; M001/M003 use different RPM and should not be called a scientific contradiction. At production scale I would consider embeddings for recall, but retain the exact-identifier guard and human review for disputed facts.

3. How do you evaluate LLM-generated summaries or content?

I separate *selection quality* from *summary quality*. For selection, I use human-reviewed persona × phase relevance labels, report Precision@5 per scenario and the macro average, and check persona divergence and phase sensitivity without tuning labels to the output. For summaries, I check that important identifiers in the text are present in authorized sources, test invalid and unsupported outputs, and inspect whether conflicts and superseded values remain accurately attributed. I would sample summaries for human review because a passing identifier check does not prove that the causal meaning is correct.

The current demo records source IDs, uses a deterministic fallback on failed checks, and deliberately summarizes conflict/update chains without an LLM choosing a winner. In production I would add reviewer rubrics for factual attribution, usefulness, and omission, then monitor feedback and regressions over time. I would not describe the current identifier check as full hallucination detection.

4. When would you use function calling versus MCP in production?

I start with the boundary of the system. If one application has a small, known set of operations, ordinary typed functions or direct function calling are easier to test, authorize, and observe. That is enough for this MVP: its pipeline is local, and adding a protocol layer would not improve the digest. I would use MCP when the same governed capabilities need to be discovered and reused across multiple model clients or tools, with clear schemas and permissions.

The tradeoff is operational complexity. MCP does not replace access control, source validation, or audit logs; it exposes capabilities that still need those controls. If EverCurrent later offered its digest or engineering knowledge as a shared tool for other assistants, I would evaluate MCP then. I would keep ranking and ACL server-owned rather than letting an agent decide them.

5. How would you handle Slack API failures?

I first classify the failure. Rate limits should honor Retry-After; transient network and server failures get bounded exponential backoff with jitter. Authentication and permission errors need operator action rather than blind retries. I would make ingestion idempotent by Slack event or message ID, track a cursor/checkpoint, and reconcile gaps with a bounded polling pass after outages. A partial channel outage must not silently look like a complete digest.

The UI should expose data freshness and source coverage; if ingestion is stale, I would show the last known digest with a clear warning or withhold a misleading update. This take-home uses a fixed synthetic snapshot, so no real Slack API, OAuth, event handling, or retry service is implemented. The design above is the next reliability step if the prototype is connected to Slack.